

Advanced SQL Injection & Role-Based Access Control (RBAC)

Advanced Databases

M.Sc. Tran Quang Toan

Faculty of Data Science and Artificial Intelligence

Bases and coordinates

1. Advanced SQL Injection (SQLi)
2. Role-Based Access Control (RBAC)
3. Hands-on Lab: Securing dvdrental

Learning Objectives

By the end of this session, you will be able to:

- **Understand the Root Cause:** Explain why SQL Injection occurs (the mixing of data and code) and how it threatens AI data pipelines.
- **Identify Advanced Threats:** Recognize how attackers use Blind SQLi and Second-Order SQLi to bypass standard defenses.
- **Apply the Principle of Least Privilege:** Configure Role-Based Access Control (RBAC) in PostgreSQL to secure database connections properly.
- **Implement Row-Level Security (RLS):** Design and enforce fine-grained data isolation policies directly within the database engine (using `dvdrental` as a model).

Recap: Isolation Levels & Concurrency

In our last session, we discussed how DBMS handles concurrent transactions. Let's recall the 4 Isolation Levels:

- **Read Uncommitted:** Vulnerable to Dirty Reads.
- **Read Committed (Postgres Default):** Prevents Dirty Reads, but allows Non-repeatable Reads.
- **Repeatable Read:** Prevents Non-repeatable Reads. (Postgres also handles Phantom Reads well here).
- **Serializable:** The safest (but slowest) level. Transactions run as if executed sequentially.

Today, we shift our focus from protecting transactions from each other, to protecting our database from external threats.

Why should AI Engineers care about DB Security?

Traditional vs. AI Perspective

- **Traditional:** Hackers steal credit cards or user credentials.
- **AI Pipeline:** Data is the "fuel" for your models. If a hacker manipulates the database, your model learns from garbage.

The Consequence: Data Poisoning

- An attacker injects false labels into the $\mathrm{training_data}$ table.
- Your Computer Vision model (e.g., self-driving car) misclassifies stop signs as speed limits.
- Weeks of expensive GPU training are completely ruined!

Advanced SQL Injection (SQLi)

The Core Issue of SQL Injection

SQL Injection is NOT a database flaw. It is an application-level flaw caused by the **mixing of Data and Executable Code**.

The Mechanism

- Developers use string concatenation to embed raw user input directly into SQL statements.
- The Database Engine cannot distinguish between ”user input” and ”SQL commands”. It blindly parses and executes everything.

Classic SQL Injection (Bypassing Staff Login)

Imagine a web portal for employees using the staff table in dvdrental.

```
-- Hacker inputs username: Mike' OR 1=1 --
SELECT * FROM staff
WHERE username = 'Mike' OR 1=1 --' AND password = 'xxx';
```

Vulnerable Authentication Query

- The ' character breaks out of the string literal.
- OR 1=1 forces the WHERE clause to evaluate to TRUE.
- The – (Postgres comment) neutralizes the password check.
- **Result:** Attacker logs in successfully without knowing the password, gaining access to rental records!

Advanced: Blind SQL Injection (Boolean-based)

Scenario: The DVD store website hides SQL error messages. If a query fails, it just shows a "404 Not Found" page.
→ *Hackers ask the server True/False (Boolean) questions via the URL.*

```
-- Original URL checking a customer: profile.php?customer_id=5
-- Attacker tests:
customer_id=5 AND 1=1 -- (Profile loads -> TRUE)
customer_id=5 AND 1=2 -- (Profile disappears -> FALSE)
```

Boolean-based SQLi on Customer Profile

Exploitation: Guessing database info character by character.

- $\text{customer_id}=5 \text{ AND } \text{substring}(\text{version}(),1,1) = \text{'P'}$
- If the profile loads normally, the hacker knows the DB is PostgreSQL!

Advanced: Time-based Blind SQLi

Harder Scenario: The server always returns the exact same webpage regardless of TRUE/FALSE. No visual clues!

→ *Use Server Response Time as a signal.*

```
-- Searching for a film with id 10, but injecting a delay
film_id=10; SELECT CASE WHEN (current_user = 'postgres')
THEN pg_sleep(10) ELSE pg_sleep(0) END; --
```

PostgreSQL Time-based Payload on Film Search

- If the film details page takes **exactly 10 seconds** to load → The attacker knows the DB connection uses the $\mathrm{postgres}$ (Superuser) role.
- They can now proceed to extract sensitive data like the $\mathrm{payment}$ table using time delays.

Second-Order SQLi (The ”Time Bomb”)

The malicious payload does not execute immediately. It waits for the right moment.

Step 1: Storage (Innocent)

A malicious user registers in the customer table with the last name:
Smith’; DROP TABLE payment; –
The system safely escapes the string and stores it. No error occurs.

Step 2: Execution (Boom!)

At the end of the month, an Admin script retrieves this name to generate a ”Top Customers” report. The script concatenates the name into a new query **without** sanitizing it.

Golden Rule: Never fully trust data, even if it comes from your own customer *table.*

Second-Order SQLi in AI Pipelines

AI Engineers often extract data for machine learning models. Let's say you are building an AI to predict film popularity based on actors.

1. **Data Collection:** Your model reads from the actor table. An attacker previously managed to insert an actor named $\mathrm{Penelope'; DELETE\ FROM\ film; --}$
2. **Pre-processing:** You write a vulnerable Python script:

```python
# Fetch actor names from DB for feature engineering
for row in actors:
    actor_name = row['first_name'] # "Penelope'; DELETE FROM film; --"

    # Fatal Flaw: Direct string formatting into a logging/update query
    cursor.execute(f"UPDATE ai_features SET processed=True WHERE actor='{actor_name}'")
```

Vulnerable Python AI Pre-processing Script

Result: Your entire film table is deleted during model training!

Bulletproof Prevention: Prepared Statements

The ultimate fix is to separate the **Parse** phase from the **Execute** phase. Let's fix our staff login.

```
-- 1. Parse (Postgres compiles the SQL template)
PREPARE staff_login (text, text) AS
SELECT * FROM staff WHERE username = $1 AND password = $2;

-- 2. Execute (Send the payload later)
EXECUTE staff_login('Mike'' OR 1=1--', 'xxx');
```

How Prepared Statements Work

Why is it secure? In Step 1, Postgres locks the Query Tree. In Step 2, the payload Mike' OR 1=1– is treated purely as a **literal string value** for the username parameter. It cannot alter the SQL logic. The login simply fails.

Defense in Depth: Application Layer

Alongside Prepared Statements, Software Engineers must implement multiple layers of security when building apps on top of dvdrental:

- **Use ORMs (Object-Relational Mapping):** Libraries like SQLAlchemy or Prisma use Prepared Statements by default when querying film or customer tables.
- **Input Validation:** If a user searches by customer_id, enforce Integer types. If they update an email, use Regex to validate the format *before* it hits the database.
- **Principle of Least Privilege:** Even if SQLi occurs, if the application user only has SELECT permissions (no DELETE or DROP), the database survives. (Covered in Part 2!)

Role-Based Access Control (RBAC)

The Principle of Least Privilege (PoLP)

Before diving into RBAC, we must understand the golden rule of database security: **The Principle of Least Privilege**.

What does it mean?

A user, program, or application should only have the bare minimum privileges necessary to perform its function.

The Biggest Mistake in Production: Connecting your web application or AI data pipeline using the `postgres` (superuser) account.

- If a SQL Injection occurs, the hacker has root access.
- They can drop databases, read local files, or execute OS commands.

Users vs. Roles in PostgreSQL

In older versions of Postgres, "Users" and "Groups" were separate. Since Postgres 8.1, they are merged into a single concept: **ROLE**.

- A **Role** can act as a user (if it has the LOGIN privilege).
- A **Role** can act as a group (if it doesn't have LOGIN, but other roles are granted membership to it).

```
-- Creates a "group" role (cannot log in)
CREATE ROLE data_analyst;

-- Creates a "user" role (can log in)
CREATE ROLE alice WITH LOGIN PASSWORD 'securePass123';
-- Equivalently: CREATE USER alice WITH PASSWORD 'securePass123';
```

Creating Roles

Role-Based Access Control (RBAC)

Instead of assigning permissions to individual users one by one (which is a nightmare to manage), we assign permissions to **Roles**, and then assign Users to those Roles.

The RBAC Workflow in dvdrental

1. Create a group role: store_clerk.
2. Create user roles for our staff: Mike and Jon.
3. Add Mike and Jon to the store_clerk group.
4. Grant SELECT, INSERT on the rental table to store_clerk.

Managing Roles in dvdrental

Let's set up the hierarchy for our DVD Rental business.

```
-- 1. Create the Group Roles
CREATE ROLE store_manager;
CREATE ROLE store_clerk;

-- 2. Create the User Roles (Staff members)
CREATE USER mike WITH PASSWORD 'mike123';
CREATE USER jon WITH PASSWORD 'jon123';

-- 3. Assign Users to Groups
GRANT store_clerk TO mike;
GRANT store_clerk TO jon;
-- (To remove: REVOKE store_clerk FROM jon;)
```

Setting up RBAC

GRANT and REVOKE Privileges

Now we define what the store_clerk role can actually do inside the dvdrental database.

```
-- Allow clerks to read the film catalog
GRANT SELECT ON film TO store_clerk;

-- Allow clerks to manage rentals and payments
GRANT SELECT, INSERT, UPDATE ON rental TO store_clerk;
GRANT SELECT, INSERT ON payment TO store_clerk;

-- Explicitly deny deletion (Just in case)
REVOKE DELETE ON rental FROM store_clerk;
```

Assigning Table Permissions

Because Mike and Jon are members of store_clerk*, they automatically inherit these permissions!*

The Limitation of Traditional RBAC

The Problem: RBAC operates at the **Table Level**.
If we $\mathrm{GRANT\ SELECT\ ON\ rental\ TO\ store_clerk}$, it means Mike can see ALL rentals, and Jon can see ALL rentals.

The Business Requirement (Data Isolation)

In a real-world multi-store setup, Staff member Mike (Store 1) should **only** be able to see and manage rentals processed by himself, or rentals belonging to Store 1. He shouldn't see Jon's data (Store 2).

*How do we restrict access at the **Row Level** without rewriting the application logic?*

The Ultimate Weapon: Row-Level Security (RLS)

Introduced in PostgreSQL 9.5, **Row-Level Security (RLS)** allows database administrators to define policies that restrict which rows are returned by normal queries or inserted/updated by data modification commands.

- **RBAC:** "Can this user access the rental table?" (Yes/No).
- **RLS:** "Which specific rows in the rental table can this user access?"

This is highly relevant for **Multi-tenant applications** (e.g., SaaS platforms) where multiple customers share the same database tables, but data must remain strictly isolated.

How RLS Works: Policies



To implement RLS, you need two steps:

1. **Enable RLS on the table:** Tell Postgres to start checking rows for this specific table.
2. **Create a Policy:** A Policy is essentially a silent WHERE clause that Postgres automatically appends to every query made by specific users.

The $\mathrm{current_user}$ Variable

Postgres has a built-in variable called $\mathrm{current_user}$ which returns the username of the currently logged-in database session. We will use this to dynamically filter rows!

Setting up RLS in dvdrental

Let's restrict the rental table so Mike and Jon can only see their own rentals.
(Assumption: DB usernames match the username column in the staff table)

```sql
-- Step 1: Enable RLS on the target table
ALTER TABLE rental ENABLE ROW LEVEL SECURITY;

-- Step 2: Create the Policy
CREATE POLICY staff_isolation_policy ON rental
FOR ALL -- Applies to SELECT, INSERT, UPDATE, DELETE
TO store_clerk -- Only applies to this role
USING (
    -- The condition that must evaluate to TRUE for the row to be visible
    staff_id = (SELECT staff_id FROM staff WHERE username = current_user)
);
```

Creating an RLS Policy

Note: staff_id *is the foreign key in the* rental *table.*

Testing the RLS Policy

What happens when Mike runs a simple query?

```
-- Switch context to Mike
SET ROLE mike;

-- Mike runs a simple SELECT ALL
SELECT * FROM rental;
```

Simulating Mike's Session

Behind the scenes, Postgres rewrites the query to: $\text{SELECT} * \text{FROM rental WHERE staff_id} = (\text{SELECT staff_id FROM staff WHERE username} = \text{'mike'});$

- Mike only sees the 8,000 rentals he processed.
- Jon's rentals are completely invisible to Mike.
- **Zero application code changes required!**

Hands-on Lab: Securing dvdrental

Hands-on Lab: Securing dvdrental

Your Role: The Lead DBA

The management of the DVD Rental company has noticed that all employees currently use the `postgres` superuser account to access the database. This is a massive security risk!

Your Mission Today:

1. **Lab 1 (RBAC):** Create a secure Role-Based Access Control system for the store cashiers.
2. **Lab 2 (RLS):** Implement Row-Level Security so that cashiers can only view their own transactions in the `payment` table.

Lab 1: Implementing RBAC (Task)

Scenario: We have two staff members in the staff table: Mike and Jon.
Requirements:

- Create a group role named cashier_role.
- Create database users for Mike and Jon (with passwords).
- Assign both users to the cashier_role.
- Grant the cashier_role permission to SELECT on the film and staff tables.
- Grant the cashier_role permission to SELECT, INSERT on the payment table.

Time limit: 10 minutes. Try writing the SQL commands on your own before looking at the solution!

Lab 1: Implementing RBAC (Solution)

```sql
-- 1. Create the Group Role
CREATE ROLE cashier_role;

-- 2. Create User Accounts
CREATE USER mike WITH PASSWORD 'mike123';
CREATE USER jon WITH PASSWORD 'jon123';

-- 3. Assign Users to the Group
GRANT cashier_role TO mike;
GRANT cashier_role TO jon;

-- 4. Assign Table Permissions to the Group
GRANT SELECT ON film, staff TO cashier_role;
GRANT SELECT, INSERT ON payment TO cashier_role;
```

Lab 2: Implementing RLS (Task)

Scenario: Currently, if Mike logs in, he can see Jon's payments. Management wants strict data isolation.

Requirements:

- Enable Row-Level Security on the payment table.
- Create a policy named isolate_cashier_payments.
- The policy should only apply to the cashier_role.
- **Logic:** A row in payment is visible IF its staff_id matches the staff_id of the currently logged-in user (Hint: match current_user with the username in the staff table).

Time limit: 15 minutes. This is challenging!

Lab 2: Implementing RLS (Solution)

```sql
-- 1. Enable RLS on the table
ALTER TABLE payment ENABLE ROW LEVEL SECURITY;

-- 2. Create the Policy
CREATE POLICY isolate_cashier_payments ON payment
FOR SELECT
TO cashier_role
USING (
    staff_id = (SELECT staff_id FROM staff WHERE username = current_user)
);

-- 3. Testing the Policy
SET ROLE mike; -- Switch context to Mike
SELECT COUNT(*) FROM payment; -- Returns 7292 (Mike's payments only)

SET ROLE jon; -- Switch context to Jon
SELECT COUNT(*) FROM payment; -- Returns 7304 (Jon's payments only)
```

RLS Configuration for Payment Table

Key Takeaways

Congratulations on securing the dvdrental database! Remember these 3 pillars of Database Security:

1. Separate Data from Code

Always use **Prepared Statements** or ORMs to prevent SQL Injection. Never trust user input.

2. Principle of Least Privilege (RBAC)

Never connect applications using the postgres superuser. Use Roles, and explicitly GRANT only what is necessary.

3. Fine-Grained Control (RLS)

Use **Row-Level Security** to isolate data within the same table. It shifts security from the application backend directly to the database engine.

Homework & Final Project Integration

Applying Security to your AI Final Project:
For your end-of-semester Advanced Database / AI project, you must implement the following security measures:

1. **No Superusers:** Your backend/API must connect to PostgreSQL using a custom role, NOT the default root user.
2. **Implement RBAC:** Create at least two roles (e.g., $\mathrm{data_scientist}$ for read-only analytics, $\mathrm{data_engineer}$ for writing data).
3. **Implement RLS (Bonus Points):** Apply an RLS policy to your training datasets so that different user groups can only access specific partitions of the data.

We will check for SQLi vulnerabilities during your project defense!